

Unit – 2 PL/SQL - 13Hrs

Introduction to PL/SQL

PL/SQL stands for “Procedural Language extensions to the Structured Query Language.” PL/SQL is Oracle Corporation’s procedural extension for SQL and the Oracle relational database. It is a high-performance, highly integrated database language.

Oracle PL/SQL is an extension of SQL language that combines the data manipulation power of SQL with the processing power of procedural language to create super powerful SQL queries. PL/SQL ensures seamless processing of SQL statements by enhancing the security, portability, and robustness of the Database.

PL/SQL means instructing the compiler ‘what to do’ through SQL and ‘how to do’ through its procedural way. Similar to other database languages, it gives more control to the programmers by the use of loops, conditions and object-oriented concepts.

Difference between SQL and PL/SQL

SQL	PL/SQL
• SQL is a single query that is used to perform DML and DDL operations.	• PL/SQL is a block of codes that used to write the entire program blocks/ procedure/ function, etc.
• It is declarative, that defines what need to be done, rather than how things need to be done.	• PL/SQL is procedural that defines how the things needs to be done.
• Execute as a single statement.	• Execute as a whole block.
• Mainly used to manipulate data.	• Mainly used to create an application.
• Interaction with a Database server.	• No interaction with the database server.
• Cannot contain PL/SQL code in it.	• It is an extension of SQL, so that it can contain SQL inside it.

Features & Advantages of PL/SQL

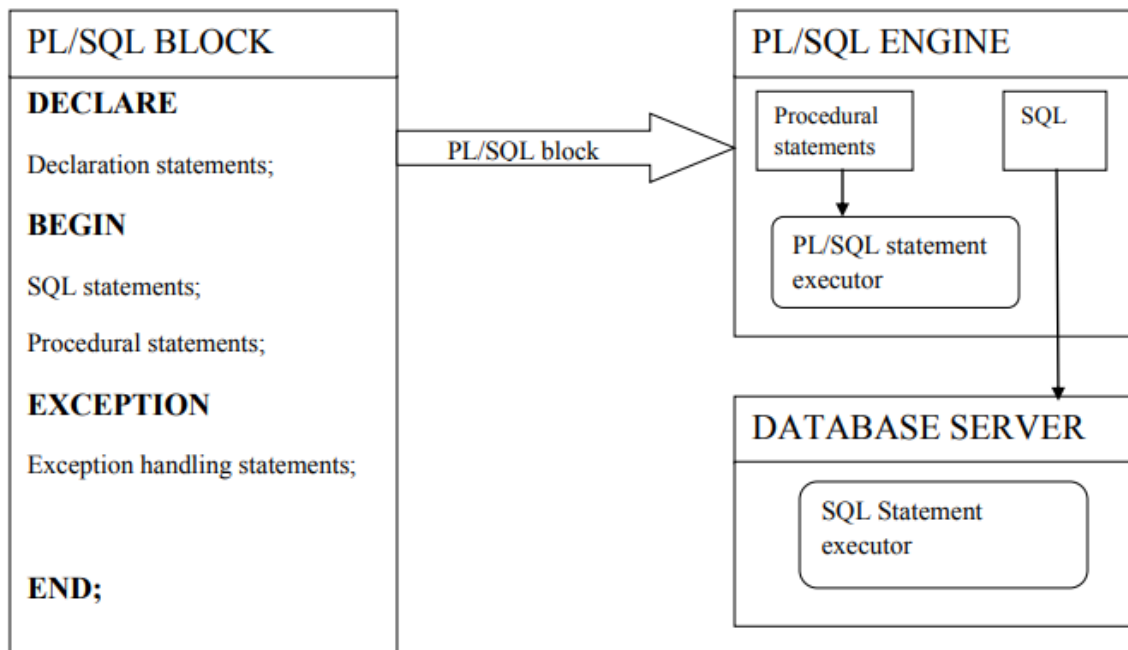
1. Better performance, as SQL is executed in bulk rather than a single statement
2. High Productivity
3. Tight integration with SQL
4. Full Portability
5. Tight Security
6. Supports Object Oriented Programming concepts.
7. Scalability and Manageability
8. Supports Web Application Development
9. Supports Server Page Development

Disadvantages of PL/SQL

1. Stored Procedures in PL/SQL uses high memory
2. Lacks functionality debugging in stored procedures
3. Any change in underlying database requires change in the presentation layer also
4. Does not completely separate roles of back-end developer and front-end developer
5. Difficult to separate HTML development with PL/SQL development

PL/SQL Architecture

PL/SQL architecture consists of the following three components, displayed in the diagram below:



PL/SQL Block:

PL/SQL block creates the structured logical blocks of code that describes the process to be executed. Such a block consists of SQL statements and PL/SQL instructions that are then passed to the Oracle engine for execution.

PL/SQL Engine:

The PL/SQL block containing the procedural statements are executed by the PL/SQL statement executor inside the PL/SQL engine. In simpler words, the PL/SQL engine provides the environment in which the PL/SQL block is executed.

Database Server:

Last but not least, comes the Database Server. This can be Oracle DB server or MySQL DB server. The DB server stores data in tables.

When an application program or a user invokes a PL/SQL block, it is submitted to the PL/SQL engine available locally. PL/SQL engine extracts the Procedural Language statements of the block. These statements are then executed separately by the PL/SQL processor. The SQL

statements in the block are passed to the SQL processor of Oracle server. These are separately executed.

PL/SQL Block Structure

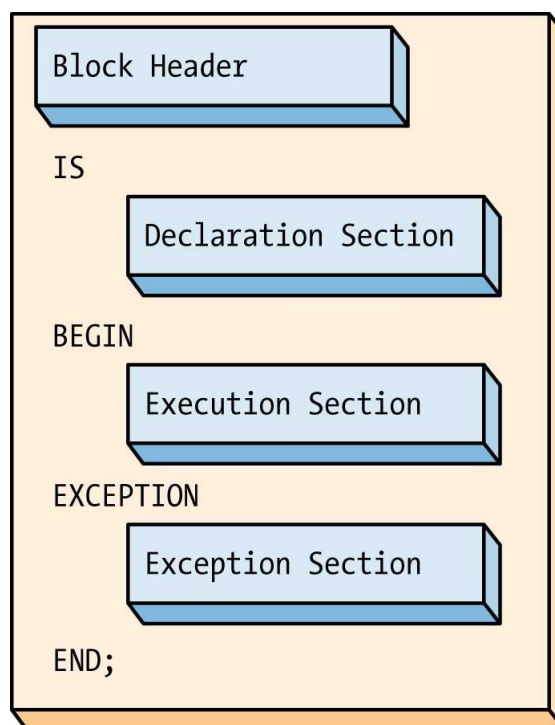
PL/SQL is a block-structured language whose code is organized into blocks. A PL/SQL block consists of three sections: declaration, executable, and exception-handling sections.

In a block, the executable section is mandatory while the declaration and exception-handling sections are optional.

A PL/SQL block has a name. Functions or Procedures is an example of a named block. A named block is stored into the Oracle Database server and can be reused later.

A block without a name is an anonymous block. An anonymous block is not saved in the Oracle Database server, so it is just for one-time use. However, PL/SQL anonymous blocks can be useful for testing purposes.

The following picture illustrates the structure of a PL/SQL block:



Header

Used only for named blocks. The header determines the way the named block or program must be called. **Optional.**

Declaration section

Identifies variables, cursors, and subblocks that are referenced in the execution and exception sections. **Optional.**

Execution section

Statements the PL/SQL runtime engine will execute at runtime. **Mandatory.**

Exception section

Handles exceptions to normal processing (warnings and error conditions). **Optional.**

Types of PL/SQL Blocks

1. Named Block
2. Anonymous Block

Anonymous blocks:

In PL/SQL, That's blocks which is not have header are known as anonymous blocks. **These blocks do not form the body of a function or triggers or procedure.**

The following example shows a simple PL/SQL anonymous block with one executable section.

```
BEGIN
  DBMS_OUTPUT.put_line ('Hello World!');
END;
```

The executable section calls the DMBS_OUTPUT.PUT_LINE procedure to display the "Hello World" message on the screen.

Named blocks:

That's PL/SQL blocks which having header or labels are known as Named blocks. **These blocks can either be subprograms like functions, procedures, packages or Triggers.**

```
CREATE OR REPLACE PROCEDURE test
IS
BEGIN
  DBMS_OUTPUT.put_line('Named Block Test');
END;
/

EXEC test;
```

PL/SQL Basics

PL/SQL Data Types

S.No	Category & Description
1	Scalar Single values with no internal components, such as a NUMBER , DATE , or BOOLEAN .
2	Large Object (LOB) Pointers to large objects that are stored separately from other data items, such as text, graphic images, video clips, and sound waveforms.
3	Composite Data items that have internal components that can be accessed individually. For example, collections and records.
4	Reference Pointers to other data items.

PL/SQL Scalar Data Types and Subtypes

S.No	Date Type & Description
1	Numeric Numeric values on which arithmetic operations are performed.
2	Character Alphanumeric values that represent single characters or strings of characters.
3	Boolean Logical values on which logical operations are performed.
4	Datetime Dates and times.

PL/SQL Numeric Data Types and Subtypes

S.No	Data Type & Description
1	PLS_INTEGER Signed integer in range -2,147,483,648 through 2,147,483,647, represented in 32 bits
2	BINARY_INTEGER Signed integer in range -2,147,483,648 through 2,147,483,647, represented in 32 bits
3	BINARY_FLOAT Single-precision IEEE 754-format floating-point number
4	BINARY_DOUBLE Double-precision IEEE 754-format floating-point number
5	NUMBER(prec, scale) Fixed-point or floating-point number with absolute value in range 1E-130 to (but not including) 1.0E126. A NUMBER variable can also represent 0
6	DEC(prec, scale) ANSI specific fixed-point type with maximum precision of 38 decimal digits

7	DECIMAL(prec, scale) IBM specific fixed-point type with maximum precision of 38 decimal digits
8	NUMERIC(pre, scale) Floating type with maximum precision of 38 decimal digits
9	DOUBLE PRECISION ANSI specific floating-point type with maximum precision of 126 binary digits (approximately 38 decimal digits)
10	FLOAT ANSI and IBM specific floating-point type with maximum precision of 126 binary digits (approximately 38 decimal digits)
11	INT ANSI specific integer type with maximum precision of 38 decimal digits
12	INTEGER ANSI and IBM specific integer type with maximum precision of 38 decimal digits
13	SMALLINT ANSI and IBM specific integer type with maximum precision of 38 decimal digits
14	REAL Floating-point type with maximum precision of 63 binary digits (approximately 18 decimal digits)

Following is a valid declaration –

```
DECLARE
    num1 INTEGER;
    num2 REAL;
    num3 DOUBLE PRECISION;
BEGIN
    null;
END;
/
```

PL/SQL Character Data Types and Subtypes

S.No	Data Type & Description
1	CHAR Fixed-length character string with maximum size of 32,767 bytes
2	VARCHAR2 Variable-length character string with maximum size of 32,767 bytes
3	RAW Variable-length binary or byte string with maximum size of 32,767 bytes, not interpreted by PL/SQL
4	NCHAR Fixed-length national character string with maximum size of 32,767 bytes
5	NVARCHAR2 Variable-length national character string with maximum size of 32,767 bytes
6	LONG Variable-length character string with maximum size of 32,760 bytes

7	LONG RAW Variable-length binary or byte string with maximum size of 32,760 bytes, not interpreted by PL/SQL
8	ROWID Physical row identifier, the address of a row in an ordinary table
9	UROWID Universal row identifier (physical, logical, or foreign row identifier)

PL/SQL Boolean Data Types

The **BOOLEAN** data type stores logical values that are used in logical operations. The logical values are the Boolean values **TRUE** and **FALSE** and the value **NULL**.

However, SQL has no data type equivalent to BOOLEAN. Therefore, Boolean values cannot be used in –

- SQL statements
- Built-in SQL functions (such as **TO_CHAR**)
- PL/SQL functions invoked from SQL statements

PL/SQL Datetime and Interval Types

The default date format is set by the Oracle initialization parameter NLS_DATE_FORMAT. For example, the default might be 'DD-MON-YY', which includes a two-digit number for the day of the month, an abbreviation of the month name, and the last two digits of the year. For example, 01-OCT-12.

Each DATE includes the century, year, month, day, hour, minute, and second. The following table shows the valid values for each field –

Field Name	Valid Datetime Values	Valid Interval Values
YEAR	-4712 to 9999 (excluding year 0)	Any nonzero integer
MONTH	01 to 12	0 to 11
DAY	01 to 31 (limited by the values of MONTH and YEAR, according to the rules of the calendar for the locale)	Any nonzero integer
HOUR	00 to 23	0 to 23
MINUTE	00 to 59	0 to 59
SECOND	00 to 59.9(n), where 9(n) is the precision of time fractional seconds	0 to 59.9(n), where 9(n) is the precision of interval fractional seconds
TIMEZONE_HOUR	-12 to 14 (range accommodates daylight savings time changes)	Not applicable
TIMEZONE_MINUTE	00 to 59	Not applicable
TIMEZONE_REGION	Found in the dynamic performance view V\$TIMEZONE_NAMES	Not applicable
TIMEZONE_ABBR	Found in the dynamic performance view V\$TIMEZONE_NAMES	Not applicable

PL/SQL Large Object (LOB) Data Types

Large Object (LOB) data types refer to large data items such as text, graphic images, video clips, and sound waveforms. LOB data types allow efficient, random, piecewise access to this data. Following are the predefined PL/SQL LOB data types –

Data Type	Description	Size
BFILE	Used to store large binary objects in operating system files outside the database.	System-dependent. Cannot exceed 4 gigabytes (GB).
BLOB	Used to store large binary objects in the database.	8 to 128 terabytes (TB)
CLOB	Used to store large blocks of character data in the database.	8 to 128 TB
NCLOB	Used to store large blocks of NCHAR data in the database.	8 to 128 TB

PL/SQL User-Defined Subtypes

A subtype is a subset of another data type, which is called its base type. A subtype has the same valid operations as its base type, but only a subset of its valid values.

PL/SQL predefines several subtypes in package **STANDARD**. For example, PL/SQL predefines the subtypes **CHARACTER** and **INTEGER** as follows –

```
SUBTYPE CHARACTER IS CHAR;  
SUBTYPE INTEGER IS NUMBER(38,0);
```

You can define and use your own subtypes. The following program illustrates defining and using a user-defined subtype –

```
DECLARE  
  SUBTYPE name IS char(20);  
  SUBTYPE message IS varchar2(100);  
  salutation name;  
  greetings message;  
BEGIN  
  salutation := 'Reader';  
  greetings := 'Welcome to the World of PL/SQL';  
  dbms_output.put_line('Hello ' || salutation || greetings);  
END;  
/
```

When the above code is executed at the SQL prompt, it produces the following result –

```
Hello Reader Welcome to the World of PL/SQL  
PL/SQL procedure successfully completed.
```

Variable Declaration in PL/SQL

The syntax for declaring a variable is –

```
variable_name [CONSTANT] datatype [NOT NULL] [:= | DEFAULT  
initial_value]
```

Example is shown below:

```
sales number(10, 2);  
pi CONSTANT double precision := 3.1415;  
name varchar2(25);  
address varchar2(100);
```

When you provide a size, scale or precision limit with the data type, it is called a **constrained declaration**. Constrained declarations require less memory than unconstrained declarations. For example –

```
sales number(10, 2);  
name varchar2(25);  
address varchar2(100);
```

Initializing Variables in PL/SQL

Whenever you declare a variable, PL/SQL assigns it a default value of NULL. If you want to initialize a variable with a value other than the NULL value, you can do so during the declaration, using either of the following –

- The **DEFAULT** keyword
- The **assignment** operator

```
counter integer := 0;  
greetings varchar2(20) DEFAULT 'Have a Good Day';
```

Example of addition of two numbers in PL/SQL:

```
DECLARE  
    a integer := 10;  
    b integer := 20;  
    c integer;  
BEGIN  
    c := a + b;  
    dbms_output.put_line('Sum: ' || c);  
END;  
/
```

Output:

```
Statement processed.  
Sum: 30
```

Taking Input From User:

```
SQL> SET SERVEROUTPUT ON;
SQL> ACCEPT X number PROMPT 'Enter any number';
Enter any number50
SQL> DECLARE
  2  a int;
  3  BEGIN
  4  a:=&X;
  5  DBMS_OUTPUT.PUT_LINE('Value :' || a);
  6  END;
  7  /
old   4: a:=&X;
new   4: a:=      50;
Value :50

PL/SQL procedure successfully completed.
```

PL/SQL Operators

PL/SQL language is rich in built-in operators and provides the following types of operators –

- Arithmetic operators
- Relational operators
- Comparison operators
- Logical operators
- String operators

Arithmetic Operators

Following table shows all the arithmetic operators supported by PL/SQL. Let us assume **variable A** holds 10 and **variable B** holds 5, then –

Operator	Description	Example
+	Adds two operands	A + B will give 15
-	Subtracts second operand from the first	A - B will give 5
*	Multiplies both operands	A * B will give 50
/	Divides numerator by de-numerator	A / B will give 2
**	Exponentiation operator, raises one operand to the power of other	A ** B will give 100000

Relational Operators

Relational operators compare two expressions or values and return a Boolean result. Following table shows all the relational operators supported by PL/SQL. Let us assume **variable A** holds 10 and **variable B** holds 20, then –

Operator	Description	Example
=	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A = B) is not true.
!= <> ~=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.

>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true

Comparison Operators

Comparison operators are used for comparing one expression to another. The result is always either **TRUE**, **FALSE** or **NULL**.

Operator	Description	Example
LIKE	The LIKE operator compares a character, string, or CLOB value to a pattern and returns TRUE if the value matches the pattern and FALSE if it does not.	If 'Zara Ali' like 'Z% A_i' returns a Boolean true, whereas, 'Nuha Ali' like 'Z% A_i' returns a Boolean false.
BETWEEN	The BETWEEN operator tests whether a value lies in a specified range. x BETWEEN a AND b means that x >= a and x <= b.	If x = 10 then, x between 5 and 20 returns true, x between 5 and 10 returns true, but x between 11 and 20 returns false.
IN	The IN operator tests set membership. x IN (set) means that x is equal to any member of set.	If x = 'm' then, x in ('a', 'b', 'c') returns Boolean false but x in ('m', 'n', 'o') returns Boolean true.
IS NULL	The IS NULL operator returns the BOOLEAN value TRUE if its operand is NULL or FALSE if it is not NULL. Comparisons involving NULL values always yield NULL.	If x = 'm', then 'x is null' returns Boolean false.

Logical Operators

Following table shows the Logical operators supported by PL/SQL. All these operators work on Boolean operands and produce Boolean results. Let us assume **variable A** holds true and **variable B** holds false, then –

Operator	Description	Examples
and	Called the logical AND operator. If both the operands are true then condition becomes true.	(A and B) is false.
or	Called the logical OR Operator. If any of the two operands is true then condition becomes true.	(A or B) is true.
not	Called the logical NOT Operator. Used to reverse the logical state of its operand. If a condition is true then Logical NOT operator will make it false.	not (A and B) is true.

Using Substitution Variables

A substitution variable allows the PL/SQL block to accept information provided by the user at the time of execution. Substitution variables are used for input purposes only. They cannot be used to output values for a user.

```
SQL> SET SERVEROUTPUT ON
SQL> DECLARE
  2  x INT :=&x;
  3  BEGIN
  4  DBMS_OUTPUT.PUT_LINE('Value of x is: '||x);
  5  END;
  6  /
Enter value for x: 30
old  2: x INT :=&x;
new  2: x INT :=30;
Value of x is: 30
```

PL/SQL procedure successfully completed.

Example – 2 Adding two numbers using substitution variables:

```
SQL> SET SERVEROUTPUT ON
SQL> DECLARE
  2  num1 INT :=&a;
  3  num2 INT :=&b;
  4  res INT;
  5  BEGIN
  6  res:=num1+num2;
  7  DBMS_OUTPUT.PUT_LINE('Sum=' || res);
  8  END;
  9  /
Enter value for a: 30
old  2: num1 INT :=&a;
new  2: num1 INT :=30;
Enter value for b: 40
old  3: num2 INT :=&b;
new  3: num2 INT :=40;
Sum=70
```

PL/SQL procedure successfully completed.

Example – 3 Finding square of a number

```
SQL> SET SERVEROUTPUT ON
SQL> DECLARE
  2  num NUMBER;
  3  res NUMBER;
  4  BEGIN
  5  num:=&num;
  6  res:=POWER(num,2);
  7  DBMS_OUTPUT.PUT_LINE('Square of number is: ' || res);
  8  END;
  9  /
Enter value for num: 5
old  5: num:=&num;
new  5: num:=5;
Square of number is: 25
```

PL/SQL procedure successfully completed.

Comments in PL/SQL

```
BEGIN
    -- single-line comment, another comment
    NULL;
    /*
        multi-line comment
        that has another single-line comment
    */
END;
/
```

Declaring Constant in PL/SQL

To declare a constant, you specify the name, **CONSTANT** keyword, data type, and the default value.

The following illustrates the syntax of declaring a constant:

```
constant_name CONSTANT datatype [NOT NULL] := expression
```

Example:

```
DECLARE
    sid CONSTANT NUMBER := 101;
BEGIN
    sid := 30; -- error
END;
```

PL/SQL Conditional Control

- **IF statements** – either execute or skip a sequence of statements based on a condition.
- **CASE statements** – choose one sequence of statements out of many possible sequences to execute.
- **GOTO** – transfer control to a labeled block or statement.
- **NULL statement** – make the code more clear.

IF Statements

IF statement allows you to either execute or skip a sequence of statements, depending on a condition. The IF statement has the three forms:

- **IF THEN**
- **IF THEN ELSE**
- **IF THEN ELSIF**

IF THEN Statement

The following illustrates the structure of the IF THEN statement:

```
IF condition THEN
    statements;
END IF;
```

Example

```
DECLARE
    age NUMBER :=21;
BEGIN
    IF age>20 THEN
        DBMS_OUTPUT.PUT_LINE('Age is greater than 20');
    END IF;
END;
/
```

Output:

```
Statement processed.
Age is greater than 20
```

IF THEN ELSE statement

```
IF condition THEN
    statements;
ELSE
    else_statements;
END IF;
```

Example:

```
DECLARE
    age NUMBER :=19;
BEGIN
    IF age>20 THEN
        DBMS_OUTPUT.PUT_LINE('Age is greater than 20');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Age is less than 20');
    END IF;
END;
/
```

Output:

```
Statement processed.
Age is less than 20
```

IF THEN ELSIF statement

```
IF condition_1 THEN
    statements_1
ELSIF condition_2 THEN
    statements_2
ELSIF condition_3 THEN
    statements_3
...
ELSE
    else_statements
END IF;
```

Example

```
DECLARE
    n_sales NUMBER := 300000;
    n_commission NUMBER( 10, 2 ) := 0;
BEGIN
    IF n_sales > 200000 THEN
        n_commission := n_sales * 0.1;
    ELSIF n_sales <= 200000 AND n_sales > 100000 THEN
        n_commission := n_sales * 0.05;
    ELSIF n_sales <= 100000 AND n_sales > 50000 THEN
        n_commission := n_sales * 0.03;
    ELSE
        n_commission := n_sales * 0.02;
    END IF;
    DBMS_OUTPUT.PUT_LINE('Comission Amount is: ' || n_commission);
END;
/
```

Output:

```
Statement processed.
Comission Amount is: 30000
```

Nested IF Statement

```
IF condition_1 THEN
    IF condition_2 THEN
        nested_if_statements;
    END IF;
ELSE
    else_statements;
END IF;
```

Example of Nested IF Statement

```
DECLARE
    sales NUMBER :=50000;
    profit NUMBER :=15000;
BEGIN
    IF sales>=50000 THEN
        IF profit>20000 THEN
            DBMS_OUTPUT.PUT_LINE('Excellent Sales!');
        ELSE
            DBMS_OUTPUT.PUT_LINE('Average Sales!');
        END IF;
    END IF;
END;
/
```

Output:

```
Statement processed.
Average Sales!
```

PL/SQL CASE Statement

The CASE statement chooses one sequence of statements to execute out of many possible sequences.

The CASE statement has two types:

- simple CASE statement and
- searched CASE statement.

Both types of the CASE statements support an optional ELSE clause.

Simple CASE statement

A simple CASE statement evaluates a single expression and compares the result with some values.

The simple CASE statement has the following structure:

```
CASE selector
WHEN selector_value_1 THEN
    statements_1
WHEN selector_value_2 THEN
    statement_2
...
ELSE
    else_statements
END CASE;
```

Simple CASE statement example

The following example compares single value (c_grade) with many possible values 'A', 'B', 'C', 'D', and 'F':

```
DECLARE
  c_grade CHAR( 1 );
  c_rank  VARCHAR2( 20 );
BEGIN
  c_grade := 'B';
  CASE c_grade
  WHEN 'A' THEN
    c_rank := 'Excellent' ;
  WHEN 'B' THEN
    c_rank := 'Very Good' ;
  WHEN 'C' THEN
    c_rank := 'Good' ;
  WHEN 'D' THEN
    c_rank := 'Fair' ;
  WHEN 'F' THEN
    c_rank := 'Poor' ;
  ELSE
    c_rank := 'No such grade' ;
  END CASE;
  DBMS_OUTPUT.PUT_LINE( c_rank );
END;
```

Output:

```
Statement processed.
Very Good
```

Searched CASE statement

The searched CASE statement evaluates multiple Boolean expressions and executes the sequence of statements associated with the first condition that evaluates to TRUE.

The searched CASE statement has the following structure:

```
CASE
WHEN condition_1 THEN statements_1
WHEN condition_2 THEN statements_2
...
WHEN condition_n THEN statements_n
[ ELSE
  else_statements ]
END CASE;
```

The following example illustrates how to use the searched CASE statement to calculate sales commission based on sales revenue.

```
DECLARE
  n_sales      NUMBER;
  n_commission NUMBER;
BEGIN
  n_sales := 150000;
  CASE
    WHEN n_sales > 200000 THEN
      n_commission := 0.2;
    WHEN n_sales >= 100000 AND n_sales < 200000 THEN
      n_commission := 0.15;
    WHEN n_sales >= 50000 AND n_sales < 100000 THEN
      n_commission := 0.1;
    WHEN n_sales > 30000 THEN
      n_commission := 0.05;
    ELSE
      n_commission := 0;
  END CASE;

  DBMS_OUTPUT.PUT_LINE( 'Commission is ' || n_commission * 100 ||
'%');
END;
```

Output:

```
Statement processed.
Commission is 15%
```

GOTO statement

GOTO statement allows you to transfer control to a labeled block or statement. The following illustrates the syntax of the GOTO statement:

GOTO label_name;

The label_name is the name of a label that identifies the target statement.

In the program, you surround the label name with double enclosing angle brackets as shown below:

<<label_name>>;

The following shows an example of using the GOTO statements.

```
BEGIN
  GOTO second_message;

  <<first_message>>
  DBMS_OUTPUT.PUT_LINE( 'Hello' );
  GOTO the_end;

  <<second_message>>
  DBMS_OUTPUT.PUT_LINE( 'PL/SQL GOTO Demo' );
  GOTO first_message;

  <<the_end>>
  DBMS_OUTPUT.PUT_LINE( 'and good bye...' );

END;
```

Output:

```
PL/SQL GOTO Demo
Hello
and good Bye...
```

NULL Statement

The PL/SQL NULL statement has the following format:

NULL;

The NULL statement is a NULL keyword followed by a semicolon (;). **The NULL statement does nothing except that it passes control to the next statement.**

The NULL statement is useful to:

- Improve code readability
- Provide a target for a GOTO statement
- Create placeholders for subprograms

The following code sends an email to employees whose job titles are Sales Representative.

```
IF job_title = 'Sales Representative' THEN
  send_email;
END IF;
```

To make it more clear, you can add a comment. For example:

```
-- Send email to only Sales Representative,
-- for other employees, do nothing
IF job_title = 'Sales Representative' THEN
  send_email;
END IF;
```

Or you can add an ELSE clause that consists of a NULL statement to clearly state that no action is needed for other employees.

```
IF job_title = 'Sales Representative' THEN
    send_email;
ELSE
    NULL;
END IF;
```

PL/SQL Repetitive Control

Basic Loop Statement

PL/SQL LOOP statement to repeat a block of code until a condition is met.

Syntax:

```
<<label>> LOOP
    statements;
END LOOP loop_label;
```

The LOOP statement executes the statements in its body and returns control to the top of the loop. **Typically, the body of the loop contains at least one EXIT or EXIT WHEN statement for terminating the loop. Otherwise, the loop becomes an infinite loop.**

The LOOP statement can have an optional label that appears at the beginning and the end of the statement.

It is a good practice to use the LOOP statement when:

- You want to execute the loop body at least once.
- You are not sure the number of times you want the loop to execute.

EXIT statement

The EXIT statement allows you to unconditionally exit the current iteration of a loop.

```
LOOP
    EXIT;
END LOOP;
```

Typically, you use the EXIT statement with an IF statement to terminate a loop when a condition is true:

```
LOOP
    IF condition THEN
        EXIT;
    END IF;
END LOOP;
```

```

DECLARE
    counter NUMBER :=0;
BEGIN
    LOOP
        counter :=counter+1;
        DBMS_OUTPUT.PUT_LINE('Raazu Poudel');
        IF counter>=2 THEN
            EXIT;
        END IF;
    END LOOP;
END;
/

```

Output:

```

Statement processed.
Raazu Poudel
Raazu Poudel

```

EXIT WHEN statement

The **EXIT WHEN** statement exits the current iteration of a loop when the condition in the **WHEN** clause is **TRUE**. Essentially, the **EXIT WHEN** statement is a combination of an **EXIT** and an **IF THEN** statement.

Syntax:

EXIT WHEN condition;

Each time the control reaches the **EXIT WHEN** statement, the condition is evaluated. If the condition evaluates to **TRUE**, then the loop terminates. Otherwise, the **EXIT WHEN** clause does nothing. Inside the loop body, you must make the condition **TRUE** at some point to prevent an infinite loop.

```

DECLARE
    counter NUMBER :=0;
BEGIN
    LOOP
        counter :=counter+1;
        DBMS_OUTPUT.PUT_LINE('Raazu Poudel');
        EXIT WHEN counter>2;
    END LOOP;
END;
/

```

Output:

```

Statement processed.
Raazu Poudel
Raazu Poudel

```

FOR LOOP statement

PL/SQL FOR LOOP executes a sequence of statements a specified number of times. The PL/SQL FOR LOOP statement has the following structure:

```
FOR index IN lower_bound .. upper_bound
LOOP
    statements;
END LOOP;
```

```
BEGIN
    FOR l_counter IN 1..5
    LOOP
        DBMS_OUTPUT.PUT_LINE( l_counter );
    END LOOP;
END;
```

Output:

```
1
2
3
4
5
```

STEP clause in FOR LOOP statement

The loop index is increased by one after each loop iteration and you cannot change the increment e.g., two, three and four. However, you can use an additional variable to simulate the increment by two, three, four, etc., as shown in the example below:

```
DECLARE
    l_step  INTEGER := 2;
BEGIN
    FOR l_counter IN 1..5 LOOP
        dbms_output.put_line (l_counter*l_step);
    END LOOP;
END;
```

Output:

```
2
4
6
8
10
```

FOR LOOP with REVERSE keyword

```
BEGIN
  FOR l_counter IN REVERSE 1..3
  LOOP
    DBMS_OUTPUT.PUT_LINE( l_counter );
  END LOOP;
END;
```

Output:

```
3
2
1
```

WHILE Loop

WHILE loop statement to execute a sequence of statements as long as a specified condition is TRUE.

Here is the syntax for the WHILE loop statement:

```
WHILE condition
LOOP
  statements;
END LOOP;
```

The following example illustrates how to use the WHILE loop statement:

```
DECLARE
  n_counter NUMBER := 1;
BEGIN
  WHILE n_counter <= 5
  LOOP
    DBMS_OUTPUT.PUT_LINE( 'Counter : ' || n_counter );
    n_counter := n_counter + 1;
  END LOOP;
END;
```

Output:

```
Counter : 1
Counter : 2
Counter : 3
Counter : 4
Counter : 5
```

WHILE loop example terminated by EXIT WHEN statement

```
DECLARE
  n_counter NUMBER := 1;
BEGIN
  WHILE n_counter <= 5
  LOOP
    DBMS_OUTPUT.PUT_LINE( 'Counter : ' || n_counter );
    n_counter := n_counter + 1;
    EXIT WHEN n_counter = 3;
  END LOOP;
END;
```

Output:

Counter : 1

Counter : 2

CONTINUE statement

The CONTINUE statement allows you to exit the current loop iteration and immediately continue on to the next iteration of that loop.

Syntax:

IF condition THEN

CONTINUE;

END IF;

The CONTINUE can be used in all loop constructs including LOOP, FOR LOOP and WHILE LOOP.

The following is a simple example of using the CONTINUE statement to skip over loop body execution for odd numbers:

```
BEGIN
  FOR n_index IN 1 .. 10
  LOOP
    -- skip odd numbers
    IF MOD( n_index, 2 ) = 1 THEN
      CONTINUE;
    END IF;
    DBMS_OUTPUT.PUT_LINE( n_index );
  END LOOP;
END;
```

Output:

2

4

6

8

10

CONTINUE WHEN statement

The CONTINUE WHEN statement exits the current loop iteration based on a condition and immediately continue to the next iteration of that loop.

The syntax of CONTINUE WHEN statement is:

CONTINUE WHEN condition;

The following example illustrates how to use the CONTINUE WHEN statement to skip over loop body execution for even numbers:

```
BEGIN
  FOR n_index IN 1 .. 10
  LOOP
    -- skip even numbers
    CONTINUE
    WHEN MOD( n_index, 2 ) = 0;
    DBMS_OUTPUT.PUT_LINE( n_index );
  END LOOP;
END;
```

Output:

```
1
3
5
7
9
```

Nested Loops

PL/SQL allows using one loop inside another loop.

The syntax for a nested basic LOOP statement in PL/SQL is as follows –

```
LOOP
  Sequence of statements1
  LOOP
    Sequence of statements2
  END LOOP;
END LOOP;
```

The syntax for a nested FOR LOOP statement in PL/SQL is as follows –

```
FOR counter1 IN initial_value1 .. final_value1 LOOP
  sequence_of_statements1
  FOR counter2 IN initial_value2 .. final_value2 LOOP
    sequence_of_statements2
  END LOOP;
END LOOP;
```

The syntax for a nested WHILE LOOP statement in PL/SQL is as follows –

```
WHILE condition1 LOOP
    sequence_of_statements1
    WHILE condition2 LOOP
        sequence_of_statements2
    END LOOP;
END LOOP;
```

The following program uses a nested basic loop to find the prime numbers from 2 to 100 –

```
DECLARE
    i NUMBER;
    j NUMBER;
BEGIN
    i := 2;
    LOOP
        j:= 2;
        LOOP
            exit WHEN ((mod(i, j) = 0) or (j = i));
            j := j +1;
        END LOOP;
        IF (j = i ) THEN
            dbms_output.put_line(i || ' is prime');
        END IF;
        i := i + 1;
        EXIT WHEN i = 50;
    END LOOP;
END;
/
```

Output:

```
2 is prime
3 is prime
5 is prime
7 is prime
11 is prime
13 is prime
17 is prime
19 is prime
23 is prime
29 is prime
31 is prime
37 is prime
41 is prime
43 is prime
47 is prime
```

PL/SQL procedure successfully completed.

Arrays in PL/SQL

PL/SQL provides a data structure called the **VARRAY**, which can store a **fixed-size sequential collection of elements of the same type**. A varray is used to store an ordered collection of data, however it is often better to think of an array as a collection of variables of the same type.

Each element in a **varray** has an index associated with it. It also has a maximum size that can be changed dynamically.

Index of varray starts with 1.

The basic syntax for creating a VARRAY type within a PL/SQL block is –

```
TYPE varray_type_name IS VARRAY(n) OF <element_type>
```

For example –

```
TYPE namearray IS VARRAY(5) OF VARCHAR2(10);  
Type grades IS VARRAY(5) OF INTEGER;
```

Example – 1

```
DECLARE  
    Type numArray IS VARRAY(5) OF INTEGER;  
    arr numArray;  
BEGIN  
    arr:=numArray(10,15,34,18,16);  
    FOR i IN 1..arr.count  
    LOOP  
        DBMS_OUTPUT.PUT_LINE(arr(i));  
    END LOOP;  
END;  
/
```

Output:

```
Statement processed.  
10  
15  
34  
18  
16
```

Example – 2

```
DECLARE
    type namesarray IS VARRAY(5) OF VARCHAR2(10);
    type grades IS VARRAY(5) OF INTEGER;
    names namesarray;
    marks grades;
    total integer;
BEGIN
    names := namesarray('Kavita', 'Pritam', 'Ayan', 'Rishav',
                        'Aziz');
    marks := grades(98, 97, 78, 87, 92);
    total := names.count;
    dbms_output.put_line('Total ' || total || ' Students');
    FOR i in 1 .. total LOOP
        dbms_output.put_line('Student: ' || names(i) || '
                              Marks: ' || marks(i));
    END LOOP;
END;
```

Output:

```
Total 5 Students
Student: Kavita Marks: 98
Student: Pritam Marks: 97
Student: Ayan Marks: 78
Student: Rishav Marks: 87
Student: Aziz Marks: 92
PL/SQL procedure successfully completed.
```

Strings in PL/SQL

PL/SQL is actually a sequence of characters with an optional size specification. The characters could be numeric, letters, blank, special characters or a combination of all. PL/SQL offers three kinds of strings –

- **Fixed-length strings** – In such strings, programmers specify the length while declaring the string. The string is right-padded with spaces to the length so specified.
- **Variable-length strings** – In such strings, a maximum length up to 32,767, for the string is specified and no padding takes place.
- **Character large objects (CLOBs)** – These are variable-length strings that can be up to 128 terabytes.

To include a single quote inside a string literal, you need to type two single quotes next to one another. For example,

```
'this isn't what it looks like'
```

Oracle database provides numerous string datatypes, such as CHAR, NCHAR, VARCHAR2, NVARCHAR2, CLOB, and NCLOB. The datatypes prefixed with an 'N' are '**national character set**' datatypes, that store Unicode character data.

If you need to declare a variable-length string, you must provide the maximum length of that string. For example, the VARCHAR2 data type. **The following example illustrates declaring and using some string variables –**

```
DECLARE
    name varchar2(20);
    company varchar2(30);
    introduction clob;
    choice char(1);
BEGIN
    name := 'John Smith';
    company := 'Infotech';
    introduction := ' Hello! I''m John Smith from Infotech.';
    choice := 'y';
    IF choice = 'y' THEN
        dbms_output.put_line(name);
        dbms_output.put_line(company);
        dbms_output.put_line(introduction);
    END IF;
END;
```

Output:

```
John Smith
Infotech
Hello! I'm John Smith from Infotech.
PL/SQL procedure successfully completed
```

PL/SQL String Functions and Operators

S.No	Function & Purpose
1	ASCII(x); Returns the ASCII value of the character x.
2	CHR(x); Returns the character with the ASCII value of x.
3	CONCAT(x, y); Concatenates the strings x and y and returns the appended string.
4	INITCAP(x); Converts the initial letter of each word in x to uppercase and returns that string.
5	INSTR(x, find_string [, start] [, occurrence]); Searches for find_string in x and returns the position at which it occurs.
6	INSTRB(x); Returns the location of a string within another string, but returns the value in bytes.
7	LENGTH(x); Returns the number of characters in x.
8	LENGTHB(x); Returns the length of a character string in bytes for single byte character set.
9	LOWER(x); Converts the letters in x to lowercase and returns that string.

10	LPAD(x, width [, pad_string]) ; Pads x with spaces to the left, to bring the total length of the string up to width characters.
11	LTRIM(x [, trim_string]); Trims characters from the left of x .
12	NANVL(x, value); Returns value if x matches the NaN special value (not a number), otherwise x is returned.
13	NLS_INITCAP(x); Same as the INITCAP function except that it can use a different sort method as specified by NLSSORT.
14	NLS_LOWER(x) ; Same as the LOWER function except that it can use a different sort method as specified by NLSSORT.
15	NLS_UPPER(x); Same as the UPPER function except that it can use a different sort method as specified by NLSSORT.
16	NLSSORT(x); Changes the method of sorting the characters. Must be specified before any NLS function; otherwise, the default sort will be used.
17	NVL(x, value); Returns value if x is null; otherwise, x is returned.
18	NVL2(x, value1, value2); Returns value1 if x is not null; if x is null, value2 is returned.
19	REPLACE(x, search_string, replace_string); Searches x for search_string and replaces it with replace_string.
20	RPAD(x, width [, pad_string]); Pads x to the right.
21	RTRIM(x [, trim_string]); Trims x from the right.
22	SOUNDEX(x) ; Returns a string containing the phonetic representation of x .
23	SUBSTR(x, start [, length]); Returns a substring of x that begins at the position specified by start. An optional length for the substring may be supplied.
24	SUBSTRB(x); Same as SUBSTR except that the parameters are expressed in bytes instead of characters for the single-byte character systems.
25	TRIM([trim_char FROM] x); Trims characters from the left and right of x .
26	UPPER(x); Converts the letters in x to uppercase and returns that string.

Example – 1

```
DECLARE
    greetings varchar2(11) := 'hello world';
BEGIN
    dbms_output.put_line(UPPER(greetings));

    dbms_output.put_line(LOWER(greetings));

    dbms_output.put_line(INITCAP(greetings));

    /* retrieve the first character in the string */
    dbms_output.put_line ( SUBSTR (greetings, 1, 1));

    /* retrieve the last character in the string */
    dbms_output.put_line ( SUBSTR (greetings, -1, 1));

    /* retrieve five characters,
       starting from the seventh position. */
    dbms_output.put_line ( SUBSTR (greetings, 7, 5));

    /* retrieve the remainder of the string,
       starting from the second position. */
    dbms_output.put_line ( SUBSTR (greetings, 2));

    /* find the location of the first "e" */
    dbms_output.put_line ( INSTR (greetings, 'e'));
END;
/
```

Output:

```
HELLO WORLD
hello world
Hello World
h
d
World
ello World
2
PL/SQL procedure successfully completed.
```

Example – 2

```
DECLARE
    greetings varchar2(30) := '.....Hello World.....';
BEGIN
    dbms_output.put_line(RTRIM(greetings, '.'));
    dbms_output.put_line(LTRIM(greetings, '.'));
    dbms_output.put_line(TRIM( '.' from greetings));
END;
/
```

Output:

```
.....Hello World
Hello World.....
Hello World

PL/SQL procedure successfully completed.
```

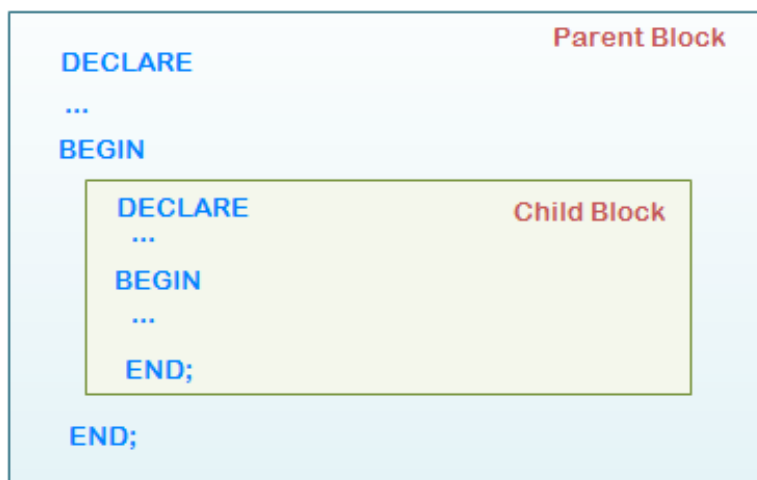
Nested Block in PL/SQL

In PL/SQL, each block can be nested into another block. They are referred as a nested block. Nested blocks are very common when we want to perform the certain process, and at the same time, the code for these process should be kept in a separate container (block).

Nested block concept will help the programmer to improve the readability by separating the complex things into each block and to handle the exception for each block inside the main outer block.

A block can be nested into another block. This can be nested either in the execution part or in the exception handling part. These block can also be labeled. One outer block can contain many inner blocks. Each inner block is once again a PL/SQL block, hence all the properties and characteristics of the inner block will be the same as outer block.

The below image gives the pictorial representation of nested block structure. Parent block is the main block and child block is the nested block.



Nested Block Syntax

```
<<outer_block>>
DECLARE
  <Declarative section>
BEGIN
  <Execution part for outer block begins>

  <<inner_block>>
  DECLARE
    <Declarative section>
```

```

BEGIN
    <Execution part for inner block>.
EXCEPTION
    <Exception handling part>
END;

    <Execution part for outer block begins>
EXCEPTION
    <Exception handling part>
END;

```

Example:

```

<<outer_block>>
DECLARE
BEGIN
    DBMS_OUTPUT.PUT_LINE('Before inner block');
    <<inner_block>>
    DECLARE
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Inside inner block');
    END;
    DBMS_OUTPUT.PUT_LINE('After inner block');
END;

```

Output:

```

Statement processed.
Before inner block
Inside inner block
After inner block

```

Note: Label is optional.

Use of DML in PL/SQL

Using Select Into

PL/SQL SELECT INTO statement is the simplest and fastest way to fetch a single row from a table into variables.

The following illustrates the syntax of the PL/SQL SELECT INTO statement:

```

SELECT
    select_list
INTO
    variable_list
FROM
    table_name
WHERE
    condition;

```

Example – 1 (Selecting a Single Column)

Consider following table with data:

Table: employee

EID	NAME	SALARY
101	Ram	30000
102	Shyam	40000
103	Hari	45000
104	Gita	50000
105	Sita	55000

```
DECLARE
  --ename VARCHAR2(30);
  --If you are uncertain about type then
  ename employee.name%TYPE;
BEGIN
  SELECT name INTO ename
  FROM employee
  WHERE eid=103;

  --displaying name
  DBMS_OUTPUT.PUT_LINE(ename);
END;
/
```

Output:

```
Statement processed.
Hari
```

```
DECLARE
  e_salary employee.salary%TYPE;
BEGIN
  SELECT MAX(salary) INTO e_salary
  FROM employee;

  --displaying name
  DBMS_OUTPUT.PUT_LINE('Max Salary is: '||e_salary);
END;
/
```

Output:

```
Statement processed.
Max Salary is: 55000
```

Example – 2 (Selecting a Complete Row)

```
DECLARE
  r_employee employee%ROWTYPE;
BEGIN
  SELECT * INTO r_employee
  FROM employee
  WHERE eid=102;

  --accessing and displaying values
  DBMS_OUTPUT.PUT_LINE('Employee Id: ' || r_employee.eid);
  DBMS_OUTPUT.PUT_LINE('Name: ' || r_employee.name);
  DBMS_OUTPUT.PUT_LINE('Salary: ' || r_employee.salary);
END;
/
```

Output:

```
Statement processed.
Employee Id: 102
Name: Shyam
Salary: 40000
```

Example – 3 (Selecting data into multiple variables)

```
DECLARE
  eid employee.eid%TYPE;
  ename employee.name%TYPE;
  esalary employee.salary%TYPE;
BEGIN
  SELECT eid,name,salary INTO eid,ename,esalary
  FROM employee
  WHERE eid=105;

  --displaying values
  DBMS_OUTPUT.PUT_LINE('Employee Id: ' || eid);
  DBMS_OUTPUT.PUT_LINE('Name: ' || ename);
  DBMS_OUTPUT.PUT_LINE('Salary: ' || esalary);
END;
/
```

Output:

```
Statement processed.
Employee Id: 105
Name: Sita
Salary: 55000
```

Using PL/SQL variable for selecting data

```
DECLARE
    id NUMBER:=102;
    myvar employee%ROWTYPE;
BEGIN
    SELECT * INTO myvar
    FROM employee
    WHERE eid=id;

    --displaying data
    DBMS_OUTPUT.PUT_LINE('Employee Id: ' || myvar.eid);
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || myvar.name);
    DBMS_OUTPUT.PUT_LINE('Employee Salary: ' || myvar.salary);
END;
/
```

Output:

```
Statement processed.
Employee Id: 102
Employee Name: Shyam
Employee Salary: 40000
```

Using Insert in PL/SQL

```
DECLARE
    id employee.eid%TYPE:=111;
    name employee.name%TYPE:='Raaju Poudel';
    salary employee.salary%TYPE:=25000;
BEGIN
    INSERT INTO employee VALUES(id,name,salary);
END;
/
```

We can prompt value from user as well using substitution variable,

```
DECLARE
    id employee.eid%TYPE:=&id;
    name employee.name%TYPE:=&name;
    salary employee.salary%TYPE:=&salary;
BEGIN
    INSERT INTO employee
    VALUES(id,name,salary);
END;
/
```

We can also insert calculated value as follows,

```
DECLARE
    basic_salary NUMBER:=25000;
    bonus NUMBER:= 5000;
    total_salary NUMBER;
BEGIN
    total_salary:=basic_salary+bonus;
    INSERT INTO employee VALUES('110', 'Raaju', total_salary);
END;
/
```

Inserting values by incrementing Id:

```
DECLARE
    e_id employee.eid%TYPE;
BEGIN
    --selecting maximum id from table
    SELECT MAX(eid) INTO e_id
    FROM employee;

    --inserting data
    INSERT INTO employee VALUES(e_id+1, 'Raaju Poudel', 34000);
END;
/
```

EID	NAME	SALARY
101	Ram	30000
102	Shyam	40000
103	Hari	45000
104	Gita	50000
105	Sita	55000
106	Raaju Poudel	34000

Using Sequence:

In Oracle, you can create an autonumber field by using sequences. A sequence is an object in Oracle that is used to generate a number sequence. This can be useful when you need to create a unique number to act as a primary key.

Sequence is a feature supported by some database systems to produce unique values on demand. Some DBMS like **MySQL** supports AUTO_INCREMENT in place of Sequence.

AUTO_INCREMENT is applied on columns, it automatically increments the column value by 1 each time a new record is inserted into the table.

Sequence is also somewhat similar to AUTO_INCREMENT but it has some additional features too.

Syntax:

```
CREATE SEQUENCE sequence_name  
MINVALUE value  
MAXVALUE value  
START WITH value  
INCREMENT BY value;
```

Create sequence as follows:

```
CREATE SEQUENCE myseq INCREMENT BY 5;
```

Generate new value as follows:

```
myseq.NEXTVAL
```

Dropping Sequence:

```
DROP SEQUENCE myseq;
```

Using sequence on Insert

Applying sequence on above table,

Create sequence as follows,

```
CREATE SEQUENCE myseq  
START WITH 107  
INCREMENT BY 1;
```

```
BEGIN  
    INSERT INTO employee  
    VALUES(myseq.NEXTVAL, 'Rajasvi Poudel', 45000);  
END;  
/
```

EID	NAME	SALARY
101	Ram	30000
102	Shyam	40000
103	Hari	45000
104	Gita	50000
105	Sita	55000
106	Raaju Poudel	34000
107	Rajasvi Poudel	45000

Using Update in PL/SQL

Let's assume we have commission column in above table,

```
ALTER TABLE employee ADD commission NUMBER;
```

EID	NAME	SALARY	COMMISSION
101	Ram	30000	-
102	Shyam	40000	-
103	Hari	45000	-
104	Gita	50000	-
105	Sita	55000	-

Let's calculate 10% commission on salary and update for eid 103.

```
DECLARE
    e_salary employee.salary%TYPE;
    e_commission NUMBER;
BEGIN
    SELECT salary INTO e_salary
    FROM employee WHERE eid=103;

    e_commission:=(e_salary*10)/100;
    --updating after calculation
    UPDATE employee SET commission=e_commission
    WHERE eid=103;
END;
/
```

EID	NAME	SALARY	COMMISSION
101	Ram	30000	-
102	Shyam	40000	-
103	Hari	45000	4500
104	Gita	50000	-
105	Sita	55000	-

Calculating commission based on condition:

- salary >= 50000 then 15%
- salary >= 40000 and < 50000 then 10%
- below 40000 then 5%

```

DECLARE
    e_salary employee.salary%TYPE;
    e_commission NUMBER;
BEGIN
    SELECT salary INTO e_salary
    FROM employee WHERE eid=104;
    --applying condition
    IF e_salary>=50000 THEN
        e_commission:=(e_salary*15)/100;
    ELSIF e_salary>=40000 AND e_salary<50000 THEN
        e_commission:=(e_salary*10)/100;
    ELSE
        e_commission:=(e_salary*5)/100;
    END IF;

    --updating after calculation
    UPDATE employee SET commission=e_commission
    WHERE eid=104;
END;
/

```

EID	NAME	SALARY	COMMISSION
101	Ram	30000	-
102	Shyam	40000	-
103	Hari	45000	4500
104	Gita	50000	7500
105	Sita	55000	-

Using loop for updating all rows:

```

DECLARE
    e_salary employee.salary%TYPE;
    e_commission NUMBER;
BEGIN
    FOR val IN 101..105
    LOOP
        --calculating equal 10% commission
        SELECT salary INTO e_salary FROM employee WHERE eid=val;
        e_commission:=(e_salary*10)/100;
        --updating after calculation
        UPDATE employee SET commission=e_commission WHERE eid=val;
    END LOOP;
END;
/

```

EID	NAME	SALARY	COMMISSION
101	Ram	30000	3000
102	Shyam	40000	4000
103	Hari	45000	4500
104	Gita	50000	5000
105	Sita	55000	5500

Calculating based on condition:

```

DECLARE
    e_salary employee.salary%TYPE;
    e_commission NUMBER;
BEGIN
    FOR val IN 101..105
    LOOP
        SELECT salary INTO e_salary FROM employee WHERE eid=val;

        --applying condition
        IF e_salary >= 50000 THEN
            e_commission := (e_salary * 15) / 100;
        ELSIF e_salary >= 40000 AND e_salary < 50000 THEN
            e_commission := (e_salary * 10) / 100;
        ELSE
            e_commission := (e_salary * 5) / 100;
        END IF;

        --updating after calculation
        UPDATE employee SET commission = e_commission WHERE eid = val;
    END LOOP;
END;
/

```

EID	NAME	SALARY	COMMISSION
101	Ram	30000	1500
102	Shyam	40000	4000
103	Hari	45000	4500
104	Gita	50000	7500
105	Sita	55000	8250

Transaction Control in PL/SQL

TCL stands for Transaction Control Language, and includes statements that are used to manage the changes that are made from DML statements. It enhances the transactional nature of Oracle SQL. **TCL commands are used to manage transactions in the database.**

The TCL commands in Oracle SQL are:

- **COMMIT:** saves any changes made to the database.
- **ROLLBACK:** undoes any changes made to the database.
- **SAVEPOINT:** creates a point in your transaction to which you can roll back to.

COMMIT

- COMMIT will save all the DML changes since the last committed state
- When the user issues COMMIT, it is known as end of transaction
- Transaction can be defined as logical unit of work
- All DDL commands are automatically commits

```
commit;  
commit emp999;
```

SAVEPOINT

- You can rollback up to SAVEPOINT
- Transaction is a unit of work, SAVEPOINT is a sub-unit of work
- You cannot COMMIT to a SAVEPOINT, commit will save all DML changes since the last committed state
- When you ROLLBACK or COMMIT, then intermediate SAVEPOINT are automatically cleared
- You can only ROLLBACK sequentially
- Within a transaction, you can have Two SAVEPOINT with the same name; the latest SAVEPOINT will supersede the previous one

```
SAVEPOINT abc;  
ROLLBACK to abc;
```

ROLLBACK

- ROLLBACK will undo the DML changes since the last committed state
- only DML commands are affected by ROLLBACK and COMMIT

```
ROLLBACK;  
ROLLBACK emp999;
```

Example:

Let's consider following table Student and data:

SID	NAME
102	Shyam
101	Ram

Let's Execute following commands:

```
INSERT INTO student VALUES(105,'Shyam');  
COMMIT;  
-- this is permanent, cannot rolled back  
  
UPDATE student SET name='Raaju' WHERE sid=105;  
SAVEPOINT A;  
  
INSERT INTO student VALUES(106,'Hari');  
SAVEPOINT B;  
  
INSERT INTO student VALUES(107,'Gita');
```

After selecting table you'll get following result:

SID	NAME
101	Ram
102	Shyam
105	Raaju
106	Hari
107	Gita

Now execute rollback as follows then select data:

```
ROLLBACK TO B;  
SELECT * FROM student;
```

SID	NAME
101	Ram
102	Shyam
105	Raaju
106	Hari

Now execute rollback once again and select data,
ROLLBACK TO A;
SELECT * FROM student;

SID	NAME
101	Ram
102	Shyam
105	Raaju

PL/SQL Example:

Let's consider following table student:

SID	NAME
101	Ram
102	Shyam

```
BEGIN
    INSERT INTO student VALUES(103,'Hari');
    COMMIT;

    INSERT INTO student VALUES(104,'Gita');
    ROLLBACK;
END;
/
```

Result after selecting data:

SID	NAME
101	Ram
102	Shyam
103	Hari

Example – 2

```
BEGIN
  UPDATE student SET name='Raaju' WHERE sid=103;
  SAVEPOINT test1;

  INSERT INTO student VALUES(111,'Gita');
  SAVEPOINT test2;

  ROLLBACK TO test1;
END;
/
```

After selecting data we will get following result:

SID	NAME
101	Ram
102	Shyam
103	Raaju